The Portfolio Game — Summer Project

Week 1 Assignment: Introduction to Game Theory

Indian Game Theory Society, IIT Kanpur

Instructions: *Submit exactly one* `.ipynb` *file with every solution, each in its own cell. Add a comment with the question number at the top of each cell.*

Question 1 — Reading Payoff Matrices

Consider the following payoff matrix for a two-player game. Player 1 chooses rows (Top / Bottom) and Player 2 chooses columns (Left / Right). Each cell shows (Payoff to P1, Payoff to P2).

```
            Left     Right
Top      (3, 3)   (0, 5)
Bottom   (5, 0)   (1, 1)
```

Answer all three parts below as Python comments in your cell (e.g. `# (a) ...`). No code required.

(a) What payoff does Player 1 receive if P1 plays Top and P2 plays Right?

(b) What payoff does Player 2 receive if both players play Bottom?

(c) If you were Player 1, which strategy gives you a higher payoff when Player 2 plays Left? When P2 plays Right?

Question 2 — Dominant Strategy Finder (2-Player)

Write a Python function that takes a payoff matrix for a two-player game and identifies whether either player has a strictly dominant strategy.

Represent the payoff matrix as a 2D list of tuples, where each tuple is (payoff_p1, payoff_p2). For example, a 2x2 game is:

```
matrix = [
    [(6, 4), (8, 1)],  # P1 plays Row 0
    [(4, 6), (7, 3)]   # P1 plays Row 1
]
```

(a) Check each of Player 1's rows: is there a row that always gives a strictly higher payoff than every other row, regardless of P2's column?

(b) Do the same for Player 2's columns.

(c) Print a clear output: which player (if any) has a dominant strategy and what it is (row/column index). If none exists, print that too.

Test your function on: (i) the Advertise/Not-Advertise matrix from the slides, (ii) the Prisoner's Dilemma matrix, and (iii) the Stag Hunt matrix.

Question 3 — IESDS Solver (2-Player)

Implement a function that runs Iterated Elimination of Strictly Dominated Strategies (IESDS) on a general m x n two-player game.

(a) Represent the game as a 2D list of (p1_payoff, p2_payoff) tuples.

(b) In each iteration, scan all remaining rows for P1 and all remaining columns for P2 to find any strictly dominated strategy.

(c) Eliminate that strategy and print which row or column was removed and why.

(d) Repeat until no more eliminations are possible.

(e) Print the surviving strategies. If a single cell remains, declare it as the predicted outcome.

Test your function on the 3x4 matrix from the slides (strategies A/B/C vs D/E/F/G). Verify that it eliminates down to (B, D).

Question 4 — Nash Equilibrium Finder: Pure Strategies (2-Player)

Write a function that finds all pure-strategy Nash Equilibria in a two-player game using the best-response method.

(a) For each column (fixed P2 strategy), find the row(s) where P1's payoff is maximised. Mark these as P1's best responses.

(b) For each row (fixed P1 strategy), find the column(s) where P2's payoff is maximised. Mark these as P2's best responses.

(c) A cell (i, j) is a Nash Equilibrium if and only if it is simultaneously a best response for both players. Collect and print all such cells.

(d) If no pure-strategy Nash Equilibrium exists, print that none was found.

Test your function on: (i) Prisoner's Dilemma, (ii) Stag Hunt, (iii) Chicken, (iv) Battle of the Sexes, and (v) the 3x3 matrix from the slides (strategies A/B/C vs X/Y/Z). Print results for each.

Question 5 — Nash Equilibrium Finder: N-Player Game

Extend your Nash Equilibrium finder to handle a game with N players, each with a finite set of pure strategies. A strategy profile is now an N-tuple, and each player's payoff depends on the full profile.

Represent the game as follows:

```
# strategies: list of lists, strategies[i] = strategy names for player i
# payoffs: dict mapping strategy profile tuple -> list of payoffs (one per
player)
# Example for 2 players:
strategies = [['C', 'D'], ['C', 'D']]
payoffs = {
    ('C','C'): [-1,-1],
    ('C','D'): [-3, 0],
    ('D','C'): [ 0,-3],
    ('D','D'): [-2,-2]
}
```

(a) Enumerate all possible strategy profiles using itertools.product.

(b) For each profile, and for each player i, check whether player i can improve their payoff by switching to any other strategy while all other players hold theirs fixed. If no player can profitably deviate, the profile is a Nash Equilibrium.

(c) Print all Nash Equilibria found, or state that none exist in pure strategies.

Test on this 3-player game. Three firms each choose Cooperate (C) or Defect (D) on a shared project. Payoffs are defined as:

```
strategies = [['C','D'], ['C','D'], ['C','D']]
payoffs = {
    ('C','C','C'): [4, 4, 4],
    ('C','C','D'): [1, 1, 6],
    ('C','D','C'): [1, 6, 1],
    ('C','D','D'): [0, 3, 3],
    ('D','C','C'): [6, 1, 1],
    ('D','C','D'): [3, 0, 3],
    ('D','D','C'): [3, 3, 0],
    ('D','D','D'): [2, 2, 2]
}
```

Run your function and print the Nash Equilibria. In 2-3 sentences, explain whether the result surprises you and what it implies about collective action.

Question 6 — Interactive Game Solver

Build an interactive game solver that allows a user to input a custom two-player payoff matrix at runtime (via keyboard input) and then automatically runs all three analyses: dominant strategy detection, IESDS, and Nash Equilibrium finding.

(a) Ask the user to enter the number of strategies for Player 1 (rows) and Player 2 (columns).

(b) Ask the user to input payoffs row by row, entering each cell as two space-separated integers (p1 payoff, then p2 payoff).

(c) Display the matrix back to the user in a readable format before running the analysis.

(d) Run all three analyses in sequence and print the results clearly labelled.

Test it by entering the Battle of the Sexes matrix manually when prompted and verify the output is correct.
